Document number: P0702R0
Date: 2017-06-18
Reply-To: Mike Spertus, Symantec (mike_spertus@symantec.com)
Audience: {Evolution, Core} Working Group

Language support for Constructor Template Argument Deduction

Introduction

This paper details some language considerations that emerged in the course of integrating Class Template Argument Deduction into the standard library as adopted in p0433r2 on which we hope to receive clarification from the committee.

List vs direct initialization

The current standard wording implies the following:

```
tuple t{tuple{1, 2}};   // Deduces tuple<int, int>
vector v{vector{1, 2}}; // Deduces vector<vector<int>>
```

We find it seems inconsistent and difficult to teach that `vector` prefers list initialization while such similar code for `tuple` prefers copy initialization. In Kona, EWG voted (wg21-only link) to prefer copy initialization, but it is not clear whether the intent was that this apply to cases like `vector` as well.

We would like EWG to clarify what was intended in this case and, if necessary, apply any change as a DR. In light of the example above as well as §11.6.4p3.8 [dcl.init.list], we recommend that the copy deduction candidate be preferred to (implicit) list initialization when initializing from a list consisting of a single element if it would deduce a type that is reference-compatible with the argument.

As a related issue, the following code due to Jonathan Wakely is rejected by Clang and g++

```
int a[1]{};
vector v{a, a, allocator<int>()};       // Compile error
vector<int> v{a, a, allocator<int>()};  // OK
```

We also suggest that this be accepted for consistency in supporting uniform initialization as the uniform initialization arguments here have different types that cannot possibly be formed into an initializer list.

Clarifying overload precedence between rvalue reference and forwarding reference

As Barry Revzin described in the Potential in temp.deduct.partial with forwarding references and deduction guides discussion thread on isocpp.org, constructor template overload depends on the currently unspecified precedence between rvalue reference and forwarding reference as in the following example taken from the thread:

```
template <typename T>
struct A {
    A(const T&); // #1
    A(T&&);      // #2
};
```

```
template <typename U>
A(U&&) -> A<double>; // #3


int main() {
    int i = 0;
    const int ci = 0;


    A a1(0);
    A a2(i);
    A a3(ci); // Unspecified whether #2 or #3 is selected
}
```

Our recommendation is that forwarding reference be preferred to rvalue reference so that it is possible to override rvalue reference constructors via a deduction guide as Richard points out in the aforementioned discussion thread.